# Homework #3

1. As an algorithm engineer in a computational imaging team, your task is to develop robust, efficient code to add informative pseudo-color to noisy microscopy images containing spherical particles of varying sizes. The experimentation team has provided you with acquired grayscale images (particles_4k.tiff) from their recent particle experiments. Submit a technical report based on following guidelines. **(50 pts)**

(1) Signal exploration & denoising
a. Pick one of the large particles, draw a straight horizontal line through its center and export the 1-D intensity profile before filtering, after 3 × 3 median, and after 5 × 5 median. Plot all three 1D intensity signal on the same axes (legend required).
b. Noise Characterization: Pick one straight horizontal line without any particles (i.e. fully background noise) and plot the histogram of the image and show the noise type by fitting corresponding noise model.

(2) Pseudo-color mapping according to grayscale values
Map the raw and denoised version image according to pixel using a continuous palette of colors that you may prefer with color bar.

(3) Segmentation and Size-Based Pseudo-Color Mapping
a. After denoising, produce a *binary* mask (0 = background, 1 = particle), you may try Ostu thresholding method to detect the circle involved. Output the denoised version of the image and the binary mask side by side for easy visual comparison.
b. Measure equivalent diameter of each particles $d_e$=sqrt(A/pi), where A is the particle area. Map every particle depending on its size to a color using a continuous palette of colors that you may prefer. Include a color bar clearly indicating the size-to-color mapping scale.
c. Plot a particle size histogram (≥ 15 bins) and print mean ± standard-deviation in the title.

(4) Time your whole pipeline on your laptop/PC and add a small timing table for each corresponding task in the report (unit: ms).

- Image loading
- Median filtering
- Signal and noise profile plotting
- Pseudo-color mapping (grayscale and size-based)
- Image segmentation
- Size measurement and histogram generation
- Total pipeline execution

2. You are asked to continue using particles_4k.tiff image from recent particle experiments. You need to understand the Discrete Fourier Transform (DFT), compare direct matrix multiplication with fast Fourier transform algorithms, and apply the method to 1-D signals and 2-D images. **(50 pts)**
(1) Derive the 1-D DFT formula and write it in matrix form:

$$\hat{x} = W_N x$$

where the DFT matrix is

$$(W_N)_{k,n} = e^{-2\pi ikn/N}.$$

Then write a Python function to construct the DFT matrix, and for $N = 8,16,32,64$ visualize the real part, imaginary part, magnitude, and phase of the DFT matrix. Briefly explain the patterns you observe.

(2) Generate the following 1-D signal:

$$x(t) = \sin(2\pi 50t) + 0.5\sin(2\pi 120t),$$

where $0 \le t < 1$.
For signal sizes

$$N = 32,64,128,256,512,1024,$$

compute the DFT using direct matrix multiplication:

$$\hat{x}_{matrix} = W_N x.$$

Record the computation time in milliseconds. For the same signal sizes, compute the DFT using fast Fourier transform. Record the computation time in milliseconds. Compare the results from the direct DFT matrix method and the FFT method by computing the relative error:

$$\epsilon = \frac{\| \hat{x}_{matrix} - \hat{x}_{fft} \|_2}{\| \hat{x}_{fft} \|_2}.$$

Make a timing table with the following columns:

- Signal size $N$
- Direct matrix DFT time
- FFT time
- Speedup
- Relative error

Plot computation time versus $N$ on a log-log plot. Fit the timing curve and show that the matrix-based DFT approximately scales as $O(N^2)$.

(3) Fourier denoising of a 1-D signal
a. First generate a clean 1-D signal composed of two sinusoidal waves:

$$x_{\text{clean}}(t) = \sin(2\pi 50t) + 0.5\sin(2\pi 120t),$$

where

$$0 \le t < 1.$$

Use $N = 1024$uniformly spaced samples:

$$t_n = \frac{n}{N}, n = 0,1,\dots,N-1.$$

Then add Gaussian noise to the clean signal:

$$x_{\text{noisy}}(t) = x_{\text{clean}}(t) + \sigma\eta(t),$$

where $\eta(t)$is standard Gaussian noise with mean 0 and standard deviation 1, and $\sigma = 2.0$. In this problem, $x_{\text{clean}}(t)$is the known noise-free reference signal, and $x_{\text{noisy}}(t)$is the corrupted signal used for Fourier denoising.
b. Compute the Fourier transform of the noisy signal using FFT.
c. Plot the clean signal, noisy signal, and Fourier power spectrum.
d. Apply threshold-based Fourier denoising by keeping only Fourier coefficients whose power is larger than a chosen threshold. Reconstruct the filtered signal using np.fft.ifft.
e. Plot the clean signal, noisy signal, and denoised signal on the same axes.
f. Compute and compare the relative errors before and after denoising:

$$e_{\text{noisy}} = \frac{\| x_{\text{noisy}} - x_{\text{clean}} \|_2}{\| x_{\text{clean}} \|_2},$$

$$e_{\text{filtered}} = \frac{\| x_{\text{filtered}} - x_{\text{clean}} \|_2}{\| x_{\text{clean}} \|_2}.$$

Discuss whether Fourier denoising improves the signal.

(4) Load one grayscale image, such as particles_4k.tiff, and crop or resize it to the following square sizes:

$$N = 64,128,256,512,1024.$$

Show that the 2-D DFT can be written using matrix multiplication form as:

$$\hat{A} = W_M A W_N^T.$$

Write a Python function to calculate the 2D DFT using matrix multiplication. Compute the 2-D DFT of each image sizes using both direct matrix multiplication and 2D FFT methods and construct a table with the following columns:

- Image size
- Direct matrix DFT time
- FFT time
- Speedup

Plot the computation time versus image size on a log-log plot. Discuss why the matrix-based 2-D DFT is much slower than fft2.